
COMPUTATIONAL ECONOMICS

Stackelberg Duopoly in Mathematica

David A. Kendrick
Ruben P. Mercado
Hans M. Amman

1 Introduction

Stackelberg quantity games are similar to Cournot games in that both are quantity competitions.¹ In the Stackelberg game, however, firms do not choose quantities simultaneously. Rather, this is a two-stage model in which a dominant firm (or Stackelberg leader) moves first by choosing its level of output in the first stage. After observing the leader's move the other firm chooses its best response output in the second stage. As we will see, the sequential play requires a different methodology and produces different results than those of the simultaneous move Cournot game.

2 The Stackelberg Leadership Model

As is typical with sequential games, we solve the Stackelberg game backward. Thus we begin to solve the problem by characterizing how the Stackelberg follower (Firm 2) responds to the Stackelberg leader's choice of quantity. An intuition for this backward approach is that in order for the Stackelberg leader to make an optimal decision about his output choice, he must first consider how the Stackelberg follower will respond to his choice in the second period. As Firm 1, our Stackelberg leader, has the same information as Firm 2, the follower, Firm 1 can solve Firm 2's optimization problem just as well as Firm 2 can. Therefore, the Stackelberg leader solves for the Stackelberg follower's reaction function and then anticipates Firm 2's second period response when making its own output choice in the first period.

In this model we use the same specification for market demand that was used in the Cournot model. See the Mathematica file `stack.nb` that, as is discussed in Appendix A, is accessed in the same way as the previous Mathematica files. The model of the Stackelberg follower's second-period behavior is identical to Firm 2's behavior in the Cournot model with only a subtle difference. Recall from the Cournot game that Firm 2's reaction function was interpreted as its best response to Firm 1's hypothesized output Q_1 . In the Stackelberg game, Firm 2 knows Q_1 for certain since it observed Q_1 at the end of stage 1. Therefore, Firm 2 responds to Firm 1's observed output Q_1 by producing

```
IN//:= Q2s = Simplify[R2[Q1] /. Solve[React2, R2[Q1]] [[1]] ]
```

```
OUT//:= ().
```

¹See Varian (1993b), pp. 448–454.

It is important to recognize that in the foregoing statement $R2[Q1]$ is a reaction function for Firm 2, named `React2`, which is the same reaction function for Firm 2 that was found in the Cournot game. Since Firm 2, the Stackelberg follower, has already observed Firm 1's output choice, $Q1$, it responds best by choosing the output $Q2s$.

The Mathematica statement takes Firm 2's reaction equation in the form

`lhs == rhs`

(left-hand side = right-hand side) named `React2` and transforms it into an expression named `Q2s` that can later be substituted into Firm 1's optimization problem. This is accomplished by defining an expression for Firm 2's output in the Stackelberg game, `Q2s`, to be equal to the variable $R2[Q1]$, where $R2[Q1]$ is replaced (`/.`) with the expression representing the solution for the variable $R2[Q1]$ from Firm 2's reaction equation, `React2`. The `Solve` command finds the expression for the variable $R2[Q1]$ and the `Simplify` command is used again to simplify the output expression. The `[[1]]` term in the input statement tells Mathematica to use the first solution found by the `Solve` command. Although there is a unique solution to the foregoing `Solve` statement, this term is still required.

After solving Firm 2's optimization problem, we step back to the first stage and solve the Stackelberg leader's (Firm 1's) optimization problem. As we did in the Cournot model, we begin by specifying the general form of Firm 1's profit function and then replace the general demand function with our specific linear demand function:

`IN//:= eqPr1 = pie1s == Q1*(P[Q1,Q2]-c1)`

`OUT//:= pie1s == Q1 (-c1 + P[Q1, Q2])`

`IN//:= eqPr1 = %/. P[Q1,Q2] -> Price`

`OUT//:= pie1s == Q1 (a - c1 - b (Q1 + Q2))`

However, at this point the model departs from the Cournot model. As Firm 1 can solve for Firm 2's best response quantity as well as Firm 2 can, Firm 1 anticipates Firm 2's reaction to any choice of $Q1$. Therefore, Firm 1 can substitute Firm 2's reaction output (which expresses Firm 2's optimal choice of $Q2$ as a function of Firm 1's quantity $Q1$) in place of $Q2$, leaving a profit function for Firm 1 that

is a function solely of its own quantity. This is accomplished with the following command:

```
IN//:= eqPr1 = Simplify[%/. Q2 -> Q2s]
```

```
OUT//:= pie1s == (())
```

The foregoing input statement redefines the equation for the profits of Firm 1 (eqPr1) to be the equation from the previous output statement with the variable Q2 replaced with the expression for Q2s that was calculated earlier. At this point the equation representing the Stackelberg leader's profits is only a function of its own quantity choice, Q1.

Observing the previous output statement, we can easily see that the Stackelberg leader's optimal choice of quantity can then be found by differentiating the foregoing profit function with respect to that firm's choice of quantity Q1. The Mathematica statement that follows uses the derivative command, D[], to differentiate Firm 1's profit equation, eqPr1, with respect to the variable Q1:

```
IN//:= focPr1 = Simplify[D[eqPr1, Q1]]
```

```
OUT//:= 0 == a/2 - c1 + c2/2 - b Q1
```

The Stackelberg leader's output choice, Q1, can then be found by solving the above first-order condition for the variable Q1. In the following statement, the Solve command finds this expression for the variable Q1, simplifies the expression, and then names the expression Q1s (the optimal quantity of Firm 1 playing the Stackelberg game). The solution to the Stackelberg game has the leader (Firm 1) choosing its optimal quantity:

```
IN//:= Q1s = Simplify[Q1 /. Solve[focPr1, Q1][[1]]]
```

```
OUT//:= ( )
```

Firm 2 (the follower) then takes Firm 1's quantity choice as given and reacts by choosing

```
IN//:= Q2s = Simplify[Q2s /. Q1 -> Q1s]
```

```
OUT//:= ( )
```

The foregoing statement simply takes the expression for Q_{2s} and replaces the variable Q_1 with our expression for Q_{1s} in terms of the model parameters.

The resulting market quantity and market price are then calculated by substituting the firms' optimal quantities into the quantity and price equations:

```
IN//:= Qstack = Q /. Q1 -> Q1s, Q2 -> Q2s
```

```
OUT//:= ()
```

```
IN//:= Pstack = Simplify[Price /. Q -> %]
```

```
OUT//:= ()
```

The syntax for the foregoing expressions corresponds to the syntax for the expressions representing Cournot market output and price found earlier.

Once again, with the market price determined, it is possible to calculate firms' profits. Firm 1's Stackelberg profits, designated pie_{1s} , is its Stackelberg output, Q_{1s} , multiplied by the difference between its unit price and unit cost ($P_{stack} - c_1$).

```
IN//:= pie1s = Simplify[Q1s*(Pstack - c1)]
```

```
OUT//:= (()())
```

Similarly, Firm 2's profits from playing a Stackelberg game, pie_{2s} , are

```
IN//:= pie2s = Simplify[Q2s*(Pstack - c2)]
```

```
OUT//:= (()())
```

3 Comparison of Cournot and Stackelberg Models

In this final section we look at a Mathematica program that considers alternative oligopoly models and asks how our model specification might affect our predicted solutions. The point of this experiment is not only to illustrate that different

models generally lead to different solutions but to impress upon the reader the importance of choosing the correct model for the industry. Does the industry you want to model have a dominant firm that appears to lead the industry? If so, it might be more appropriate to model this industry as a Stackelberg oligopoly rather than a Cournot oligopoly. Do the firms in the industry produce a nearly homogeneous good? If not, neither the Cournot nor the Stackelberg model is likely to be an appropriate choice for modeling.

For the purposes of this experiment, we look at a duopoly industry with the same linear demand curve as in the previous programs. The only variation is that in the current model we make one more simplifying assumption. In addition to our previous assumption that firms have constant marginal costs of production, we now assume that these costs are the same for both firms ($c_1 = c_2 = c$). Because of this assumption, the reader should be warned that our purpose is not to propose any quantitative differences between the models (although some qualitative differences do become apparent). Rather our purpose is to drive home the message that modeling oligopoly markets is an art and a science. Modeling is the science. Choosing the right model is the art.

We investigate this special case of symmetric costs (i.e., marginal costs are the same constant number for all firms) by considering three alternative models' solutions. Our benchmark solution is the collusive monopoly outcome. Unlike the noncooperative Cournot and Stackelberg games, the collusive outcome assumes that the two firms agree to behave as a monopoly industry by restricting market output to the monopoly level, with each firm producing half of the monopoly market output thereby splitting monopoly market profits. We use this as our benchmark because it is the most profitable possible outcome. We then return to our now familiar models of Cournot and Stackelberg competition and ask: How do the predictions from these models differ from each other and how different are they from the collusive outcome?

In order to account for the collusive outcome, we take a slightly different approach to the general setup for this program than in the other Mathematica programs:

```
In[]:= SetAttributes[a, Constant]
```

```
SetAttributes[b, Constant]
```

```
SetAttributes[c, Constant]
```

```
IN//:= Clear[a,b,c,temp1,temp2];
```

```
Clear[Q,Q1,Q2,eqPr1,eqPr2,focPr1,focPr2,R2,pie1c,pie2c];
```

```
IN//:= Price = a - b*(Q) ;
```

Note that the setup still contains the SetAttributes commands, the Clear commands to clear any previously stored values, and our specification of our inverse demand relationship. Noticeably absent, however, is the definition of market quantity as the sum of Firm 1's output and Firm 2's output ($Q = Q_1 + Q_2$). This omission is intentional because we first want to consider the collusive (monopoly) outcome. This requires that we model the industry not as a duopoly, but rather as a monopoly and then divide the monopoly outcome between the firms.

As noted previously, solving for the collusive outcome amounts to solving for the monopoly market output and then distributing this output evenly between the firms. To accomplish this, we develop an expression for the monopolists profit function:

```
IN//:= eqPrM = ProfitM == Q(P[Q] - c) ;
```

```
eqPrM = Expand[%/. P[Q] -> Price]
```

```
OUT//:= ProfitM == Q (a - c - b Q)
```

We differentiate the profit function with respect to the choice variable, quantity (Q),

```
IN//:= focPrM = D[eqPrM, Q]
```

```
OUT//:= 0 == a - c - 2 b Q
```

and then solve for the profit-maximizing market quantity, naming this quantity Qm:

```
IN//:= Qm = Simplify[Q/. Solve[focPrM,Q][[1]]]
```

```
OUT//:= ()
```

As we assume that this collusive market output is distributed evenly between

the duopolists, we simply divide the market output, Q_m , by two in order to find each firm's output:

$$IN//:= Q1m = Qm/2$$

$$OUT//:= ()$$

$$IN//:= Q2m = Qm/2$$

$$OUT//:= ()$$

We complete the collusive solution section of the code by calculating the collusive market price, P_{mon} , and the collusive profits of one of the representative firms ($pie1m$). As both firms are assumed to split the market evenly, the two firms have equal profits:

$$IN//:= P_{mon} = \text{Simplify}[\text{Price} /. Q \rightarrow Qm]$$

$$OUT//:= ()$$

$$IN//:= pie1m = \text{Simplify}[Q1m*(P_{mon} - c)]$$

$$OUT//:= (())$$

Before continuing on to the Cournot and Stackelberg models we need to insert the definition for total market quantity that we omitted earlier:

$$IN//:= Q = Q1 + Q2 ;$$

The next two sections of code model the Cournot game and then the Stackelberg duopoly game. These models and their associated codes are nearly identical to the Cournot and Stackelberg models considered earlier with the noted exception that the constant marginal cost of production is now the same for both firms (and is denoted c in the code).

In the last section of this code we take a closer look at the solutions to the models of our symmetric cost industry and make some analytic and graphic comparisons among the collusive, the Cournot, and the Stackelberg outcomes.

The first set of comparisons considers the overall size of the market that the al-

ternative models predict. The N in the first input statement that follows is a Mathematica operator to return a numerical value, and, by the precedence rules, the term in the first input statement is $(Q_{\text{cour}}/Q_m) - 1$:

IN ::= sizeQc = N[(Qcour/Qm-1)*100]

OUT ::= 33.3333

IN ::= sizeQs = N[(Qstack/Qm-1)*100]

Out ::= 50.

The first statement in the foregoing compares the size of the Cournot market output in our symmetric cost model with the collusive outcome and shows that in the Cournot model output is 33.3 percent larger than the most profitable collusive market size. The second input statement makes a similar comparison between the Stackelberg market output and the collusive market size and shows that the Stackelberg market output is even larger—50 percent larger than the collusive market size.

The next set of statements compares the size of the individual firms' outputs predicted by the alternative models:

IN ::= sizeQ1c = N[(Q1c/Q1m-1)*100]

OUT ::= 33.3333

IN ::= sizeQ2c = N[(Q2c/Q2m-1)*100]

OUT ::= 33.3333

IN ::= sizeQ1s = N[(Q1s/Q1m-1)*100]

OUT ::= 100.

IN ::= sizeQ2s = N[(Q2s/Q2m-1)*100]

OUT ::= 0

The first two input statements above compare Firm 1's and Firm 2's market output

from the Cournot game against their collusive output levels and show that both produce 33.3 percent more in the Cournot model than the collusive outcome. The last two input statements make similar comparisons of the firms' market outputs from the Stackelberg game against their collusive output. In our symmetric costs industry we find that the Stackelberg leader (Firm 1) produces twice the collusive output and the Stackelberg follower (Firm 2) produces the same output as in the collusive game.

At this point we have shown that the collusive (monopoly) outcome has the smallest market size and therefore the highest market price. Similarly it is now clear that the Stackelberg model has the largest market size and consequently the lowest market price. What is still unclear is how industry and firms' profits compare. The first set of Mathematica statements that follow compare Cournot profits to the collusive shared monopoly profits:

```
IN//:= sizePie1c = N[(pie1c/pie1m-1)*100]
```

```
OUT//:= -11.1111
```

```
IN//:= sizePie2c = N[(pie2c/pie2m-1)*100]
```

```
OUT//:= -11.1111
```

Here we find that the Cournot profits for both firms are 11.1 percent lower than the profits that would be achieved if the firms were to collude by acting as a monopolist.

The next set of statements compares the Stackelberg profits to collusive profits:

```
IN//:= sizePies = N[((pie1s + pie2s)/(2*pie1m)-1)*100]
```

```
OUT//:= -25.
```

```
IN//:= sizePie1s = N[(pie1s/pie1m-1)*100]
```

```
OUT//:= 0
```

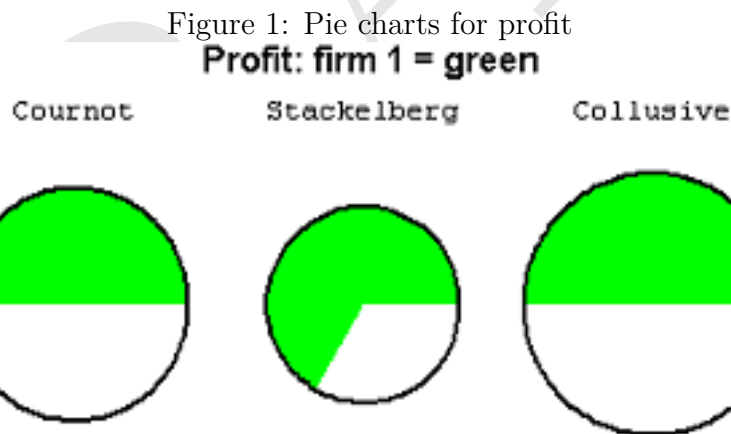
```
IN//:= sizePie2s = N[(pie2s/pie2m-1)*100]
```

```
OUT//:= -50.
```

The first statistic (which we call `sizePies`) compares industry profits of the Stackelberg and collusive outcomes. The other input statements compare the profits between these alternative outcomes for Firm 1 and Firm 2, respectively. Note that industry profits for this Stackelberg game are 25 percent lower than the monopoly profit level for the industry. Therefore, industry profits are lower in the Stackelberg game than they were in the Cournot game. However, despite having less industry profit in the Stackelberg game, we find that the Stackelberg leader (Firm 1) is able to achieve higher profits than in the Cournot game. In fact, in this symmetric cost industry, we find that the Stackelberg leader earns the same profits as in the collusive outcome.

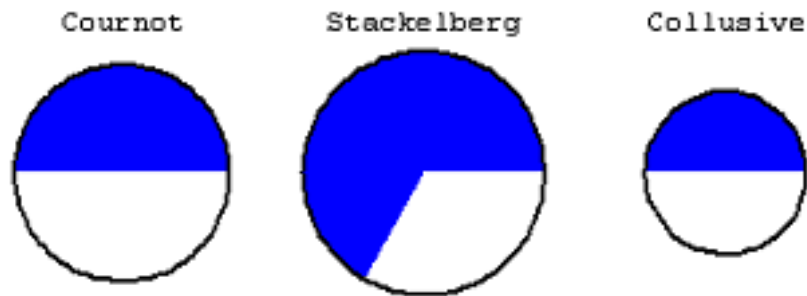
Given that collusive and Stackelberg profits are the same for Firm 1, why does the Stackelberg leader not produce at half the monopoly output? Because in such a one-shot game, Firm 2 would cheat and produce more than its collusive share. It is left to the reader to show what quantity Firm 2 would produce after observing Firm 1 produce half the monopoly market output.

The foregoing results are also summarized in the Mathematica program with a series of pie charts.



The Mathematica code that generates these sets of pie charts and the plots that follow is longer and more complicated than the code statements used throughout this chapter and so are omitted here to avoid confusion. The blue and green colors referred to in Figure 10.1 appear when the computer program is run but do not show here in the book. The darker portions of circles above correspond to blue

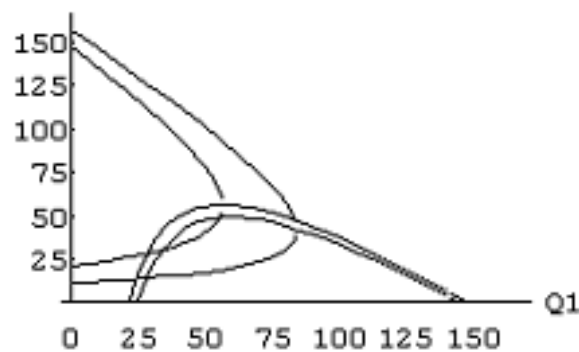
Figure 2: Pie charts for output
OUTPUT: firm 1 = Blue



and green and the lighter portions correspond to white. However, all of the code is presented in the files and the adventurous reader is encouraged to study it with the aid of a Mathematica reference manual.

The program `combine.nb` concludes with a series of plots illustrating the solutions to the Cournot and Stackelberg games. The first of these plots (Fig. 10.2) represents the firms' isoprofit curves. This is a contour mapping (similar to a geographic map) that shows how the two firms' profits vary for different combinations of their output.

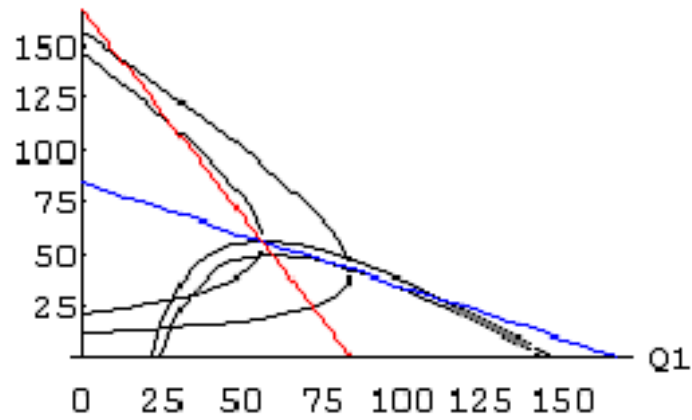
Figure 3: Contour maps of two firms' profits
Q2



Each isoprofit curve represents a different level of profits and the level of profits increases as the curves shift toward their respective axis. In the plot in Figure 10.2, Firm 2's isoprofit curves (the ones further away from the respective axis) are red and Firm 1's isoprofit curves are blue. The colors do not show here in the book but do appear when the computer program is run.

In the final plot, the isoprofit curves are combined with the plot of the reaction functions that we considered earlier.

Figure 4: Isoprofit curves combined with reaction function



4 Experiments

In this chapter we develop a series of experiments that cover many of the aspects of the models presented. However, one set of experiments is to consider modeling alternative market structures.

Appendix

A Running Mathematica

Mathematica is a widely available commercial software system. A web site for information about it is [<http://www.wolfram.com>].

Choose Programs from the Start menu and then choose Mathematica. Wait for a few seconds until a new window with a menu bar in its upper part appears. The content of this window is a white sheet called Notebook, which is like a document in a standard word processor. If you specified a file when opening Mathematica, this file is displayed.

Select Getting Started from the Help menu (located in the upper-right corner of the Mathematica window) and read the information that appears in the Info dialog box. To begin practicing with Mathematica, perform the calculations suggested in the section Doing Calculations. While doing this, you will appreciate the basic way of working with Mathematica, that is, your Notebook successively displays your inputs and the corresponding outputs. You may also notice that on the right side of your Notebook, a hierarchy of brackets appears. Each of them defines a cell (or a group of cells) that is the basic unit of organization in a Notebook. As you will quickly realize, cells can be hierarchically arranged (as in set and subsets). There are different kinds of cells: they can contain text, Mathematica input, Mathematica output, or graphs, and so on. Different small characters within each bracket identify the kind of cell. Here are some basic things you can do with cells:

- To edit a cell (i.e., to be able to work with it) just click on its bracket;
- To edit a group of cells, just click and drag on their brackets;
- To find out or change the kind of cell, edit the cell, select Style in the main menu and choose your option.
- To divide or merge cells, take the cursor to the division/insertion point of your choice, select Cell in the main menu and choose your options.
- To run a portion of a program contained in a cell or group of cells, just edit the cells and select Action-Evaluate in the main menu (or just press Shift-Enter)
- Finally, to save your Notebook, select File in the main menu and choose your options.

- Go to the book web site at [<http://www.eco.utexas.edu/compeco>] and then to the *Input Files for Chapters in the Book* section of the web site. Right click on the Leontief.nb file name and select the Save Target As ... option in order to save the file in your preferred directory.
- Go to the File menu and click Open to open the file.
- To run an input command or a cell containing a series of commands, click on the bracket on the right of it and hit Shift + Return (hold the shift key and hit Return at the same time). The output is displayed following the input, *unless* there is a ; at the end of the input command line (; suppresses the output). You can run multiple cells by highlighting the corresponding brackets with your mouse and hitting Shift + Return once.
- Modify commands and rerun them sequentially, cell after cell, so that you can see the changes in the corresponding outputs.
- If you either select the outermost bracket and press Shift-Enter or go to the Kernel menu, choose Evaluation, and Evaluate Notebook, you rerun the complete program. If your program is large this may take a few minutes and it may be difficult for you to track down the results of your modifications. On the other hand, sometimes your modifications may require an updating of previous results, a clearing of previous values, or a change of attributes (and the Clear command or the SetAttributes command are usually at the beginning of the program). In these cases you may need to rerun the complete program to avoid errors or spurious results.